НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ «КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Лабораторна робота № 2

з дисципліни «**Оброблення надвеликих масивів даних**»

Тема: «Обробка текстових даних в Apache Spark»

Виконав:
студент групи ІП-з41мп
Дишкант Лариса Леонідівна

Перевірив:
к.т.н. доцент кафедри ІПІ
Олійник Ю.О.

Київ 2024

Мета роботи: ознайомитись з методами аналізу текстів в Apache Spark

Для підготовленого набору даних засобами Apache Spark виконати завдання:

1. Нормалізація та попередня обробка даних.
    a. Провести очищення текстових даних від стоп-слів/тегів/розмітки;
    b. Виконати токенізацію текстових елементів;
    c. Провести лематизацію текстових елементів (наприклад можна використати бібліотеку Spacy - приклад роботи за ***посиланням***). Зберегти результат в окремий файл.
    d. Створити Bag of Words для всіх нормалізованих слів. Зберегти результат в окремий файл.
    e. Порахувати метрику TF-IDF для 10 слів, що найчастіше зустрічаються в корпусі документів;
    f. Порахувати косинусну подібність між полями датасету title та body за ***посиланням***.
2. Інтелектуальний аналіз текстів :
    - провести сантимент аналіз (визначення емоційної тональності – позитивний / негативний) для даних ukr_text.csv. Для визначення тональності можна використати як методи на основі словника тональності (посилання) так і методи машинного навчання.
    - провести категоризацію (визначення категорій тексту) даних методом LSA/LDA. Приклади роботи з морфоаналізатором PyMorphy наведено за ***посиланням***.

Провести очищення текстових даних від стоп-слів/тегів/розмітки

Для очищення текстових даних, завантажимо файл та видалимо спецсимволи та стопслова. Результат збережемо у файл з назвою cleaned_text.csv (рисунок 1)

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, regexp_replace, lower

spark = SparkSession.builder.appName("TextNormalization").getOrCreate()

# Завантаження даних
df = spark.read.csv("ukr_text.csv", header=True, inferSchema=True)

# Очищення текстових даних
df_cleaned = df.select(
    col("title"),
    col("body"),
    regexp_replace(lower(col("body")), "[^a-zA-Za-яА-Я0-9іеїёґ\s]", "")  # Видалення спецсимволів
)

# Збереження очищених даних
df_cleaned.write.csv("cleaned_text.csv", header=True)
```

Рисунок 1 - Завантаження та очищення даних

Виконати токенізацію текстових елементів

```
from pyspark.ml.feature import RegexTokenizer
from pyspark.sql import functions as F

# Токенізація тексту
tokenizer = RegexTokenizer(inputCol="body", outputCol="tokens", pattern="\\s+")
df_tokenized = tokenizer.transform(df_cleaned)

# Конвертація масиву токенів у рядок
df_tokenized_string = df_tokenized.withColumn("tokens", F.concat_ws(" ", "tokens"))

# Запис у CSV
df_tokenized_string.write.csv("tokenized_text.csv", header=True)
```

Рисунок 2 - Токенізація

Рисунок 3 - Результат токенізації у вигляді створеного файлу.

Провести лематизацію текстових елементів (наприклад можна використати бібліотеку Spacy - приклад роботи за *посиланням*). Зберегти результат в окремий файл.

```
# Встановлюємо Spacy
!pip install -U spacy

# Завантажуємо українську модель
!python -m spacy download uk_core_news_sm
```

Рисунок 4 - Встановлення Spacy

Для роботи потрібно встановити Spacy та завантажити очищених та токенізованих даних. Після чого проводимо лемитизацію та зберігаємо результат у файл (рисунок 5 та рисунок 6).

```
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType
# Завантаження очищених і токенізованих даних
df = spark.read.csv("tokenized_text.csv", header=True, inferSchema=True)

# Завантаження української моделі SpaCy
nlp = spacy.load("uk_core_news_sm")

# Функція для лематизації тексту
def lemmatize_text(text):
    doc = nlp(text)
    lemmas = [token.lemma_ for token in doc]
    return " ".join(lemmas)

# Реєстрація UDF для лематизації
lemmatize_udf = udf(lemmatize_text, StringType())

# Додавання нової колонки з лематизованим текстом
df_lemmatized = df.withColumn("lemmas", lemmatize_udf(col("tokens")))

# Збереження результату в CSV
df_lemmatized.select("title", "lemmas").write.csv("lemmatized_text2.csv", header=True)
```

Рисунок 5 - Лематизація

Рисунок 6 - Результат лематизації

Створити Bag of Words для всіх нормалізованих слів. Зберегти результат в окремий файл

```python
from pyspark.sql import SparkSession
from pyspark.ml.feature import CountVectorizer
from pyspark.sql.functions import col, split, udf
from pyspark.sql.types import StringType

# Ініціалізація Spark
spark = SparkSession.builder.appName("BagOfWords").getOrCreate()

# Завантаження лематизованих даних
df = spark.read.csv("lemmatized_text2.csv", header=True, inferSchema=True)

# Розбивка лематизованого тексту на токени
df_tokens = df.withColumn("tokens", split(col("lemmas"), " "))

# Ініціалізація CountVectorizer для створення Bag of Words
vectorizer = CountVectorizer(inputCol="tokens", outputCol="features")
model = vectorizer.fit(df_tokens)

# Трансформуємо дані для створення Bag of Words
df_bow = model.transform(df_tokens)

# Конвертація SparseVector у текстовий формат
def vector_to_string(vector):
    return " ".join([f"{i}:{v}" for i, v in zip(vector.indices, vector.values)])

vector_to_string_udf = udf(vector_to_string, StringType())

# Додавання стовпця з текстовим представленням Bag of Words
df_bow_string = df_bow.withColumn("features_str", vector_to_string_udf(col("features")))

# Збереження Bag of Words у файл
df_bow_string.select("title", "features_str").write.csv("bag_of_words.csv", header=True)

# Збереження словника (всі унікальні слова)
vocab = model.vocabulary
with open("vocabulary.txt", "w", encoding="utf-8") as f:
    for word in vocab:
        f.write(word + "\n")
```

Рисунок 7 - Створення мішка слів та збереження його у файлі

Рисунок 8 - Результат створення мішка слів

a. Порахувати метрику TF-IDF для 10 слів, що найчастіше зустрічаються в корпусі документів;

Порахувати косинусну подібність між полями датасету title та body за ***посиланням***.

```
tokenizer_body = Tokenizer(inputCol="body", outputCol="words_body")

words_df_title = tokenizer_title.transform(spark_df)
words_df_body = tokenizer_body.transform(words_df_title)

# Використання HashingTF для перетворення тексту в вектори
hashingTF_title = HashingTF(inputCol="words_title", outputCol="features_title", numFeatures=10000)
hashingTF_body = HashingTF(inputCol="words_body", outputCol="features_body", numFeatures=10000)

featurized_title = hashingTF_title.transform(words_df_body)
featurized_body = hashingTF_body.transform(featurized_title)

# Обчислення IDF
idf_title = IDF(inputCol="features_title", outputCol="tfidf_title")
idf_body = IDF(inputCol="features_body", outputCol="tfidf_body")

idf_model_title = idf_title.fit(featurized_body)
idf_model_body = idf_body.fit(featurized_body)

tfidf_title = idf_model_title.transform(featurized_body)
tfidf_body = idf_model_body.transform(tfidf_title)

# Витягування векторів
title_vector = tfidf_title.select("tfidf_title").first()[0].toArray()
body_vector = tfidf_body.select("tfidf_body").first()[0].toArray()

# Обчислення косинусної подібності
cosine_sim = cosine_similarity([title_vector], [body_vector])

print(f"Косинусна подібність між 'title' та 'body': {cosine_sim[0][0]}")
```

```
Косинусна подібність між 'title' та 'body': 0.0
```

Рисунок 9 - Косинусна подібність

Провести сантимент аналіз (визначення емоційної тональності – позитивний / негативний) для даних ukr_text.csv. Для визначення тональності можна використати як методи на основі словника тональності (посилання) так і методи машинного навчання.

```python
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

# Ініціалізація Spark сесії
spark = SparkSession.builder.appName("Sentiment Analysis").getOrCreate()

# Завантаження даних
df = spark.read.csv("ukr_text.csv", header=True)

# Словник для сентимент-аналізу
positive_words = set(['добрий', 'позитивний', 'чудовий'])  # Приклади
negative_words = set(['поганий', 'негативний', 'гіркий'])  # Приклади

# Функція для отримання тональності
def sentiment_analysis(text):
    if not text:
        return None
    words = set(text.split())
    positive_count = len(words.intersection(positive_words))
    negative_count = len(words.intersection(negative_words))
    if positive_count > negative_count:
        return 'позитивний'
    elif negative_count > positive_count:
        return 'негативний'
    else:
        return 'нейтральний'

# Необхідно імплементувати UDF для функції
sentiment_udf = F.udf(sentiment_analysis)

# Додати стовпець з тональністю для поля 'Body'
df = df.withColumn("sentiment", sentiment_udf(df["Body"]))
df.select("Body", "sentiment").show()
```

Рисунок 10 - Визначення тональності

```
+--------------------+-----------+
|                Body|  sentiment|
+--------------------+-----------+
|"Київ - перспекти...|нейтральний|
|"   З'явилося від...|нейтральний|
|У Києві на Бессар...|нейтральний|
|Київ вночі 16 сер...|нейтральний|
|Уночі Київ вкотре...|нейтральний|
|"Колишній народни...|нейтральний|
|"У Києві троє нев...|нейтральний|
|У Києві на вихідн...|нейтральний|
|"На майдані Незал...|нейтральний|
|У Києві у зв'язку...|нейтральний|
|У вересні набудут...|нейтральний|
|"Поліція затримал...|нейтральний|
|"Увечері, 20 липн...|нейтральний|
|"Прес-служба Київ...|нейтральний|
|"У Києві під час ...|нейтральний|
|"Німецького музик...|нейтральний|
|"Про це повідомля...|нейтральний|
|"Акція за участю ...|нейтральний|
|"У мережі опублік...|нейтральний|
|"У Києві дві ночі...|нейтральний|
+--------------------+-----------+
only showing top 20 rows
```

Рисунок 11 - Результат визначення тональності

Провести категоризацію (визначення категорій тексту) даних методом LSA/LDA. Приклади роботи з морфоаналізатором PyMorphy наведено за ***посиланням***.

```python
spark = SparkSession.builder.appName("LDA Example").getOrCreate()

# Завантаження даних
df = spark.read.csv("ukr_text.csv", header=True)

# Токенізація тексту
tokenizer = Tokenizer(inputCol="Body", outputCol="words")
words_data = tokenizer.transform(df)

# Видалення стоп слів
custom_stop_words = [
    "і", "а", "в", "на", "з", "ді", "до", "з",
    "не", "це", "всі", "як", "так", "буде", "якщо",
    "вона", "він", "ми", "ви", "того", "от", "по",
    "що", "за", "коли", "але", "для", "то", "взагалі", "у", "від", "та", "про", "під"

]

stop_words_remover = StopWordsRemover(inputCol="words", outputCol="words_without_stops", stopWords=custom_stop_words)
words_data = stop_words_remover.transform(words_data)

# Обчислення Count Vectorizer
count_vectorizer = CountVectorizer(inputCol="words_without_stops", outputCol="features")
cv_model = count_vectorizer.fit(words_data)
vectorized_data = cv_model.transform(words_data)

# Модель LDA
lda = LDA(k=5, seed=1, optimizer="em", featuresCol="features")
lda_model = lda.fit(vectorized_data)

# Топ-слова для кожної теми
topics = lda_model.describeTopics(3)
vocab = cv_model.vocabulary

# Виведення топ-слів для кожної теми
for topic in topics.collect():
    print("Тема:")
    for word_idx in topic.termIndices:
        print(f"  {vocab[word_idx]}")
```

Рисунок 12 - Категоризація тексту

```
Тема:
  -
  –
  повідомляє
Тема:
  –
  раніше
  індекс
Тема:

  -
  +
Тема:
  –
  області
  року
```

Рисунок 13 - Результат категоризації тексту

Висновок: На лабораторній роботі ознайомитись з методами аналізу текстів в Apache Spark.